

RAPPORT FINAL UNIQUE — BIPROJECT QA

Projet : Assurance Qualité & Automatisation - BiProject

Binôme : Youssef Chaari & Mohamed Aziz Zouari

Date : 15 Mars 2026

Version : 1.3 (Master Consolidé pour Remise PDF)

1. Introduction

1.1 Contexte Académique

Ce rapport s'inscrit dans le cadre du module de Validation et Test Logiciel. Il documente l'intégralité de la démarche d'assurance qualité appliquée à l'application **BiProject**, couvrant les analyses statiques et dynamiques, ainsi que l'automatisation des tests sur l'ensemble de la pyramide logicielle.

1.2 Objectif du Projet QA

L'objectif principal est de garantir la robustesse, la sécurité et la maintenabilité du système via une stratégie de test rigoureuse. La mission combine la détection proactive de vulnérabilités critiques (IDOR, Broken Access Control) et la mise en place d'une suite de non-régression automatisée performante.

1.3 Rappel du SUT (System Under Test)

Le système sous test est **BiProject**, une plateforme de gestion commerciale intégrant un catalogue de produits, une gestion de paniers et des statistiques métier (Analytics).

2. Présentation du projet BiProject

2.1 Objectif Métier

BiProject permet aux utilisateurs de consulter des produits, de passer des commandes et de suivre leurs transactions. Une interface d'administration permet la gestion du catalogue et l'analyse des indicateurs de performance (KPIs).

2.2 Architecture Technique

- **Backend** : API REST construite avec **ASP.NET Core 8**, utilisant Entity Framework Core pour la persistance des données.
- **Frontend** : Application Single Page (SPA) développée en **Angular**, utilisant des gardes de route (AuthGuard) pour la sécurité client.

2.3 Rôles Utilisateurs

- 1. **User** (Standard) : Consultation, achat et gestion de ses propres commandes.
- 2. **Admin** : Gestion complète du catalogue et accès aux données stratégiques de Business Intelligence.

3. Plan et stratégie de test

3.1 Stratégie Retenue

Nous avons adopté une approche hybride combinant :

- **Analyse Statique** : Inspections manuelles et revues de code pour détecter les failles logiques.
- **Tests Dynamiques Automatisés** : Suivant la pyramide des tests pour optimiser la couverture et le temps d'exécution.

3.2 Pyramide des Tests

- 1. **Tests Unitaires (Base)** : Validation de la logique métier isolée (Backend) — 3 tests.
- 2. **Tests d'Intégration (Cœur)** : Validation des contrats d'API et du contrôle d'accès réel via `WebApplicationFactory` — 11 tests.
- 3. **Tests Système E2E (Sommet)** : Validation des parcours utilisateurs complets via Selenium + POM — 6 tests.

4. Tests statiques (Détails)

L'analyse statique identifie des défauts structurels précoces sans exécution de code.

4.1 Méthodologie

- **Code Review** : Inspection manuelle sur `AuthService.cs` et `OrderController.cs`.
- **Analyse Sécurité** : Focus sur l'OWASP A01:2021 (Broken Access Control).

4.2 Registre des Anomalies Détectées

ID	Localisation	Anomalie	Gravité	Recommandation
AN-01	<code>OrderController.cs</code>	IDOR (Accès direct aux commandes tiers)	 Critique	Valider le <code>UserId</code> via Claims JWT
AN-02	DTO Création Order	Trust Boundary Violation (UserId falsifiable)	 Majeure	Utiliser le <code>UserId</code> du token exclusivement
AN-03	<code>AuthService.cs</code>	Absence de logging des échecs d'auth	 Mineure	Injecter <code>ILogger</code> pour l'audit

5. Catalogue des Cas de Test (Résumé Technique)

Le catalogue complet comprend **20 cas de test uniques**.

5.1 Authentification & Inscription

- **CT-01 à CT-06** : Login (Pass/Fail), génération de Token JWT, rejet des mots de passe invalides. (☑ Pass)
- **CT-10, CT-11** : Inscription utilisateur et gestion des doublons via l'UI Selenium. (☑ Pass)

5.2 Catalogue & Commandes

- **CT-03, CT-07 à CT-09** : Protection des accès (AuthGuards et 401 Unauthorized API). (☑ Pass)
- **CT-12, CT-13, CT-15** : Consultation catalogue et création de produits (Admin). (☑ Pass)
- **CT-16, CT-17** : Consultation des commandes propres et processus de Checkout. (☑ Pass)

5.3 Sécurité Étendue (Preuves par l'échec)

- **CT-14** : Interdiction de création de produit pour le rôle 'User'. (☑ Pass)
- **CT-18** : Isolation des commandes (Sécurité anti-IDOR fixe). (✗ Fail)
- **CT-19** : Limitation des privilèges Analytics (BI) pour les Users. (✗ Fail)
- **CT-20** : Blocage de l'accès à la commande d'un autre utilisateur (IDOR dynamique). (✗ Fail)

6. Automatisation & Maintenabilité

6.1 Patterns Utilisés

- **AAA (Arrange, Act, Assert)** : Structure standard de tous les tests xUnit.
- **POM (Page Object Model)** : Utilisé pour les tests Selenium afin de désolidariser les sélecteurs CSS de la logique de validation.
- **IClassFixture** : Gestion optimisée du cycle de vie du serveur de test Backend.

7. Résultats d'exécution détaillés

7.1 Synthèse par Couche

Couche	Tests exécutés	Pass	Fail	Succès %
Unitaires (Backend)	3	3	0	100%
Intégration (Backend)	11	8	3	72%
Système (Selenium)	6	6	0	100%
TOTAL	20	17	3	85%

7.2 Détail des Échecs Révélateurs

Son échec (HTTP 200 reçu au lieu de 403) constitue la preuve formelle du défaut de contrôle d'accès :

- **CT-20** : Validation IDOR dynamique (Accès unauthorized 200 OK).
- **CT-19** : Accès User aux KPIs Admin (Données BI exposées).
- **CT-18** : Confirmation de la vulnérabilité IDOR sur identifiant fixe.

8. Analyse approfondie des anomalies (Preuves par l'échec)

Ce projet démontre l'utilisation des tests automatisés comme **outils d'audit de sécurité**. L'échec des tests CT-18, 19 et 20 est volontaire et documenté pour justifier les correctifs prioritaires.

9. Matrice de Traçabilité

Ce tableau relie les exigences aux cas de test implémentés.

Exigence	Description	CT ID	Résultat
REQ-AUTH-01	Authentification sécurisée (JWT)	CT-01, 02, 04, 05, 06	☑ Pass
REQ-AUTH-02	Protection des routes UI (AuthGuard)	CT-03	☑ Pass
REQ-AUTH-03	Inscription de nouveaux comptes	CT-10, 11	☑ Pass
REQ-SEC-01	Protection des Endpoints API (401/403)	CT-07, 08, 09, 14	☑ Pass
REQ-SEC-02	Protection contre l'accès IDOR	CT-18, CT-20	✗ Fail
REQ-SEC-03	Isolation des données Analytics (Admin)	CT-19	✗ Fail
REQ-CAT-01	Consultation du catalogue de produits	CT-12, 13	☑ Pass
REQ-ORD-01	Processus d'achat complet (Checkout)	CT-17	☑ Pass

10. Procédure complète d'exécution

10.1 Commandes d'exécution

```
# 1. Démarrer l'API (Port 5120)
cd BiProject.Api && dotnet run
```

```
# 2. Démarrer l'UI (Port 4200)
cd BiProject.Ui && npm run start

# 3. Lancer les tests Backend (xUnit)
dotnet test BiProject.Tests

# 4. Lancer les tests Selenium (Pytest)
cd tests_selenium && python -m pytest tests/ -v
```

11. Limites du projet

- **Environnement** : Dépendance au "Seeding" initial de la base de données.
- **Sécurité** : Les vulnérabilités sont maintenues pour démonstration pédagogique.

12. Bonus : Prototype Robot Framework

Un prototype basé sur **Robot Framework** a été réalisé et **validé par exécution séparée** (2 scénarios passés). Il démontre la polyvalence de l'automatisation proposée.

13. Déclaration d'usage de l'IA

Ce projet a été réalisé en mode pair-programming avec l'IA **Antigravity**. L'humain a conservé le contrôle total sur la stratégie, le design des cas de test et la validation des résultats.

14. Conclusion

Le projet BiProject QA présente une maturité de test avancée. La combinaison de la pyramide d'automatisation et de la détection proactive de vulnérabilités critiques via l'analyse statique et dynamique répond aux exigences majeures de qualité. Le projet est conforme et prêt pour la remise académique.

15. Annexes

- **Annexe A** : Catalogue détaillé ([cas_de_test.md](#)) — Contient les prérequis et étapes pas-à-pas.
- **Annexe B** : Rapports de Logs ([Build_Tests_Logs.txt](#)) — Traces d'exécution brutes.